

Teoría de Algoritmos

(75.29 / 95.06) Curso Ing. Podberezski

Trabajo Práctico Final

7 de Julio de 2025

Grupo: **BigO**

Apellido y Nombre	Padrón	e-mail
Czerwiak, Juan Joaquin	109.640	jczerwiak@fi.uba.ar
Huzan, Hugo Javier	67.910	hhuzan@fi.uba.ar
Rivero, Julián Agustín	112.454	jarivero@fi.uba.ar
Rojo, Santiago	110.700	srojo@fi.uba.ar

Parte 1: Personal para proyectos.....	3
Análisis del problema.....	3
Forma estándar.....	3
Forma de holgura.....	3
Simplex.....	3
Complejidad Temporal y Espacial.....	4
Solución Paso a Paso.....	5
Análisis.....	7
Programación Lineal Entera.....	7
Pseudocódigo.....	8
Pasos para la ejecución del programa.....	10
Parte 2: Buscando una solución aceptable.....	11
Análisis del Problema.....	11
Algoritmos.....	12
Algoritmo 1: ratio dinero/conflictos.....	12
Pseudocódigo.....	12
Algoritmo 2: randomizado Montecarlo.....	14
Pseudocódigo.....	14
Análisis empírico de las soluciones.....	16
Parte 3: Autómatas finitos.....	20
Lenguajes simples.....	20
Contiene subcadena “1011”.....	20
Número par de ceros.....	21
Poseen un 1 en posiciones impares.....	22
Termina con tres 1.....	24
Lenguajes Complejos.....	25
Contiene subcadena “1011” y número par de ceros.....	25
Poseen un 1 en posiciones impares y finalizan con tres 1.....	27
Referencias.....	29

Parte 1: Personal para proyectos

Una consultora ofrece personal calificado para 3 tipos de proyectos. Cada proyecto requiere ingenieros y diseñadores. Actualmente tiene en su nómina 17 y 11 de estos profesionales respectivamente. El proyecto tipo “A” requiere 1 ingeniero y 3 diseñadores. El proyecto tipo “B” requiere 1 ingeniero y 2 diseñadores. Finalmente el proyecto tipo “C” requiere 3 y 2. Por cada proyecto gana respectivamente 200, 100 y 300 (en millones de pesos). Desean determinar cuántos proyectos y de qué tipo aceptar para maximizar las ganancias.

Análisis del problema

La información del enunciado se resume en la siguiente tabla:

PROYECTO	INGENIEROS (17)	DISEÑADORES (11)	GANANCIA
A	1	3	200
B	1	2	100
C	3	2	300

Forma estándar

Sean x_1, x_2, x_3 variables de decisión que representan la cantidad de proyectos tipo A, B y C respectivamente, se propone un problema de programación lineal cuya función objetivo es $z = 200x_1 + 100x_2 + 300x_3$ y se encuentra sujeto a las restricciones:

Los ingenieros requeridos no deben superar el total: $x_1 + x_2 + 3x_3 \leq 17$

Los diseñadores requeridos no deben superar el total: $3x_1 + 2x_2 + 2x_3 \leq 11$

Toda cantidad no puede ser negativa: $x_1, x_2, x_3 \geq 0$

Forma de holgura

A partir de la forma estándar, transformamos las restricciones a variables de holgura; resultando en un problema equivalente:

Maximizar

$$z = 200x_1 + 100x_2 + 300x_3$$

Sujeto a:

$$x_4 = 17 - x_1 - x_2 - 3x_3 \text{ (Cantidad de ingenieros)}$$

$$x_5 = 11 - 3x_1 - 2x_2 - 2x_3 \text{ (Cantidad de diseñadores)}$$

Simplex

Un algoritmo conocido que resuelve cualquier problema de programación lineal real con variables en forma de slack es Simplex. Este algoritmo resuelve el problema al ir aumentando la cantidad del tipo de

proyecto que más aumenta¹ la función objetivo hasta el límite descrito por las restricciones del problema. Cuando ningún proyecto aumenta la ganancia obtenida hasta el momento, se termina el proceso y se retorna un vector con las cantidades de los proyectos que maximizan la función objetivo.

```
Inicializar tablero
Repetir mientras existe un coeficiente positivo en la función objetivo
    Seleccionar una variable de salida `vs` entre las variables básicas
    con mayor coeficiente positivo en la función objetivo

    Determinar la variable de entrada `ve` cuya restricción es la que
    ajusta a `vs`

    Si la variable `vs` no se ajusta
        Retornar `solución ilimitada`

    Realizar el proceso de pivot entre `vs` y `ve`

Obtener ganancia
Reconstruir solución

Retornar solución y ganancia
```

Podemos obtener la ganancia total al aplicar las variables de decisión obtenidas a la función objetivo.

Dado que no hicimos transformaciones de no negatividad, cambio de desigualdad ni de igualdad a desigualdad (ya que el problema se pudo modelar sin necesidad de los mismos); las variables resultantes tendrán la información que buscamos, cada variable representando lo definido previamente.

En nuestro caso tendremos un vector $(x_1, x_2, x_3, x_4, x_5)$ que representarán la cantidad de proyectos A, B, C a aceptar y la cantidad de ingenieros y diseñadores sobrantes respectivamente.

Complejidad Temporal y Espacial.

Para el análisis de complejidad llamaremos n a las variables de decisión y m a las restricciones.

Vamos a analizar una implementación de simplex básica.

Inicializar el tablero de $(m + 1) \times (n + m + 1)$ donde:

- La primera fila representa la función objetivo con los coeficientes de las $n + m$ variables, donde las m variables de holgura tienen coeficiente 0. Y donde la última fila posee la ganancia inicializada en 0.
- Las m filas restantes representan las restricciones, de misma manera que la primer fila con la excepción que las variables de holgura inician con 1 en la restricción que corresponda y la última columna es el término independiente de la restricción.

¹ El algoritmo Simplex en sí no requiere elegir la variable que más aumente la función objetivo, simplemente una variable cuyo coeficiente sea positivo. En este caso decidimos que sea la de mayor coeficiente porque tiende a reducir iteraciones.

Luego se repite los siguientes pasos pasando por cada punto del politopo hasta que no se pueda maximizar más la función objetivo.

1. Seleccionar la variable saliente tomará un costo lineal respecto a las variables de decisión, ya que solamente hay que fijarse el coeficiente de las n variables de decisión de la primera fila de la tabla.
2. Seleccionar la variable entrante tomará un costo lineal respecto a la cantidad de restricciones del problema al verificar el ratio *coeficiente de $vs \div$ término independiente* de cada restricción y seleccionar aquel posea el mínimo positivo.
3. El proceso de pivot llevará tiempo $(m + 1) * (m + n + 1)$ al tener que actualizar toda la tabla con operaciones matemáticas básicas luego de reemplazar la variable saliente por la entrante.

La ganancia se obtiene en tiempo constante de la primera fila y última columna de la tabla.

La reconstrucción se realiza en tiempo $m \cdot (m + n)$ al recorrer cada restricción y fijarse cual es la variable con valor 1. A la misma le corresponderá el valor del término independiente.

Dado lo detallado, la complejidad espacial queda establecida por la tabla en $O(m(m+n))$ y la temporal por la cantidad de iteraciones k por el proceso de pivot, es decir:

$$S(n, m) = O(m^2 + mn)$$

$$T(n, m) = O(k(m^2 + mn))$$

Algunas implementaciones de simplex mejoran levemente la complejidad temporal y/o espacial teórica, como Simplex por Reducción que posee $S(n, m) = O(m^2 + n)$. A contraparte, estos algoritmos son más complejo de analizar y normalmente solo brindan ventajas a problemas de mayor tamaño

Esto, aunque no lo parezca a simple vista, resulta en una complejidad no polinomial dado que k es la cantidad de puntos en el politopo, y estos puntos, en el peor de los casos es exponencial. Sin embargo, en la práctica la mayoría de estos puntos no forman parte del espacio de soluciones y otros no son recorridos por Simplex al recorrer por los puntos que más maximicen la función objetivo.

Solución Paso a Paso

Inicialización

MAXIMIZAR

$$z = 200x_1 + 100x_2 + 300x_3$$

RESTRICCIONES:

$$x_4 = 17 - x_1 - x_2 - 3x_3$$

$$x_5 = 11 - 3x_1 - 2x_2 - 2x_3$$

$$x_1, x_2, x_3, x_4, x_5 \geq 0$$

SOLUCIÓN PARCIAL:

x_1	x_2	x_3	x_4	x_5	VALOR OBJETIVO
0	0	0	17	11	0

Iteración 1

Elegir variable saliente:

$$z = 200x_1 + 100x_2 + 300x_3 \rightarrow x_3 \text{ es la de mayor coeficiente}$$

Elegir variable entrante:

$$x_4 = 17 - x_1 - x_2 - 3x_3 \rightarrow x_3 = \frac{17}{3} \simeq 5,67$$

$$x_5 = 11 - 3x_1 - 2x_2 - 2x_3 \rightarrow x_3 = \frac{11}{2} = 5,5$$

$$\Rightarrow x_5 \text{ es la que más restringe a } x_3$$

Proceso de pivot:

Despejar variable entrante en la fila pivote

$$x_5 = 11 - 3x_1 - 2x_2 - 2x_3$$

$$\Rightarrow x_3 = \frac{11}{2} - \frac{3}{2}x_1 - x_2 - \frac{1}{2}x_5$$

Reemplazar variable entrante en demás restricciones

$$x_4 = 17 - x_1 - x_2 - 3x_3$$

$$\Rightarrow x_4 = 17 - x_1 - x_2 - 3\left(\frac{11}{2} - \frac{3}{2}x_1 - x_2 - \frac{1}{2}x_5\right)$$

$$\Rightarrow x_4 = 17 - x_1 - x_2 - \frac{33}{2} + \frac{9}{2}x_1 + 3x_2 + \frac{3}{2}x_5$$

$$\Rightarrow x_4 = \frac{1}{2} + \frac{7}{2}x_1 + 2x_2 + \frac{3}{2}x_5$$

Reemplazar variable entrante en función objetivo

$$z = 200x_1 + 100x_2 + 300x_3$$

$$\Rightarrow z = 200x_1 + 100x_2 + 300\left(\frac{11}{2} - \frac{3}{2}x_1 - x_2 - \frac{1}{2}x_5\right)$$

$$\Rightarrow z = 200x_1 + 100x_2 + 1650 - 450x_1 - 300x_2 - 150x_5$$

$$\Rightarrow z = 1650 - 250x_1 - 200x_2 - 150x_5$$

MAXIMIZAR

$$z = 1650 - 250x_1 - 200x_2 - 150x_5$$

RESTRICCIONES:

$$x_4 = \frac{1}{2} + \frac{7}{2}x_1 + 2x_2 + \frac{3}{2}x_5$$

$$x_3 = \frac{11}{2} - \frac{3}{2}x_1 - x_2 - \frac{1}{2}x_5$$

$$x_1, x_2, x_3, x_4, x_5 \geq 0$$

SOLUCIÓN PARCIAL:

x_1	x_2	x_3	x_4	x_5	VALOR OBJETIVO
0	0	5.5	0.5	0	1650

Todos los coeficientes de la función objetivo son negativos. Por lo tanto ya no se puede seguir mejorando.

Reconstrucción del resultado

La “solución” hallada implica que la ganancia se podría maximizar a 1650 millones de pesos cuando:

- Se aceptan:
 - **5.5** proyectos de tipo “C”
 - Ningún proyecto tipo “A” o “B”
- Se asignan todos los diseñadores
- Y queda sin asignar **0.5** Ingeniero

Análisis

¿Corresponde a la solución óptima del problema? Según la naturaleza del problema, ¿lo puede encontrar en tiempo polinomial? Justificar.

La solución encontrada por Simplex **no es óptima** por contener valores reales **no enteros** para las variables que cuantifican los proyectos aceptados y los recursos no asignados.

Esto está dado porque Simplex se basa en buscar la solución en los vértices del politopo factible, y esos vértices no necesariamente tienen coordenadas enteras.

Programación Lineal Entera

Investigar la aplicación de Branch and Bound dentro de programación lineal entera. Explique cómo funciona paso a paso. Brinde pseudocódigo. ¿Lo puede utilizar en el problema? En caso afirmativo desarrolle como hacerlo paso a paso.

Paso 1: Resolver el “problema relajado”

- Se ignora temporalmente que las variables deben ser enteras
- Se resuelve con Simplex
- Si el problema no es factible o es peor a la mejor encontrada, se poda
- Si la solución es entera, la solución es óptima.
- Si no es entera
 - Obtener y aplicar cota inferior al truncar las variables
 - Proceder al paso 2

Paso 2: Branch

- Se elige la primera variable no entera de la solución.

- Se crean dos subproblemas:
 - el “subproblema izquierdo” con la restricción adicional $x_i \leq \text{floor}(x_i^*)$
 - el “subproblema derecho” con la restricción adicional $x_i \geq \text{ceiling}(x_i^*)$

Esto divide el espacio de soluciones en 2 partes que eliminan el valor no entero actual de x_i

Paso 3: Bound

- Se resuelve cada problema con Simplex (nueva relajación sin restricción de enteros)
- En cada subproblema:
 - Si Simplex no devuelve una solución válida, se descarta.
 - Si es entera:
 - Si es mejor, se actualiza como la mejor solución global.
 - Si no es entera, se evalúa si su valor objetivo es mejor que la mejor solución entera actual(cota)
 - Si no se puede mejorar se descarta (poda)
 - Si se puede mejorar, se ramifica nuevamente

Pseudocódigo

```
Sea mejor_valor = -infinito
Sea mejor_solucion = ∅

Branch&Bound(coeficientes, restricciones):

    solucion, valor = ResolverPorSimplex(coeficientes, restricciones)

    Si No hay solucion:
        Retornar mejor_solucion, mejor_valor

    Si valor ≤ mejor_valor:
        // Poda por cota: ya es peor que lo mejor conocido
        Retornar mejor_solucion, mejor_valor

    Si todas las variables de solucion son enteras:
        Si valor > mejor_valor:
            mejor_valor = valor
            mejor_solucion = solucion
        Retornar mejor_solucion, mejor_valor

    Obtener cota inferior al evaluar la función objetivo con las variables
    truncadas.
    Si la cota inferior es mejor a la mejor solución, actualizar

    Seleccionar una variable no entera

    piso = floor(variable)
    techo = ceil(variable)

    nueva_restriccion_izq = [  $x_i \leq \text{piso}$  ]
```



```
restricciones_izq = restricciones + [nueva_restriccion_izq]
solucion_izq, valor_izq = Branch&Bound(coeficientes, restricciones_izq)
Si valor_izq > mejor_valor:
    actualizar mejor_valor y mejor_solucion

nueva_restriccion_der = [ x_i ≥ techo]
restricciones_der = restricciones + [nueva_restriccion_der]
solucion_der, valor_der = Branch&Bound(coeficientes, restricciones_der)
Si valor_der > mejor_valor:
    actualizar mejor_valor y mejor_solucion

Retornar mejor_solucion, mejor_valor
```

Ejemplo paso a paso:

Problema inicial:

Relajación lineal: $x = [0, 0, 5.5] \rightarrow Z = 1650$

Como $x_3 = 5.5$ no es entero, ramificamos

Primera ramificación (x_3):

Subproblema A: $x_3 \leq 5$

Subproblema B: $x_3 \geq 6$

- Explorando Subproblema A ($x_3 \leq 5$):

Relajación: $x = [0.33, 0, 5] \rightarrow Z = 1566.67$

Como $x_1 = 0.33$ no es entero, ramificamos

Segunda ramificación (x_1):

Subproblema A1: $x_1 \leq 0$

Subproblema A2: $x_1 \geq 1$

- Explorando Subproblema A1 ($x_1 \leq 0$):

Relajación: $x = [0, 0.5, 5] \rightarrow Z = 1550$

Como $x_2 = 0.5$ no es entero, ramificamos

Tercera ramificación (x_2):

- Explorando Subproblema A1a ($x_2 \leq 0$):

Relajación: $x = [0, 0, 5] \rightarrow Z = 1500$

Encontramos la primera solución entera, **mejor = 1500**

- Explorando Subproblema A1b ($x_2 \geq 1$):

Relajación: $[0, 1, 4.5]$ con $Z = 1450$. Como $1450 < 1500$, **se poda**

- Explorando Subproblema A2 ($x_1 \geq 1$):

Relajación: $x = [1, 0, 4] \rightarrow Z = 1400$. Como $1400 < 1500$, **se poda**

- Explorando Subproblema B ($x_3 \geq 6$):

Inviabile ya que necesitamos 18 ingenieros y 12 diseñadores para completar 6 proyectos de tipo C

Resultado: $x = [0, 0, 5]$ con $Z = 1500$

Pasos para la ejecución del programa

1. Instalar python

2. Crear entorno virtual:

```
python -m venv env
```

3. Activar entorno:

- En Linux/macOS: `source env/bin/activate`

- En Windows: `.\env\Scripts\activate`

4. Instalar dependencias:

```
pip install pulp
```

5. Ejecutar programa:

```
python proyecto.py
```

Parte 2: Buscando una solución aceptable.

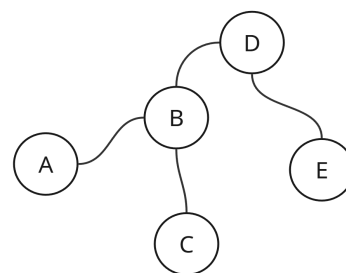
Para el desarrollo de un proyecto estamos buscando inversores. Contamos con una preselección de “n” de estos. cada uno de ellos nos prometió aportar cierta cantidad de dinero (este valor no necesariamente es igual para cada inversor). Nos gustaría aceptar a todos. Sin embargo, algunos inversores se niegan a participar si otros están. Nos gustaría determinar a cuáles aceptar para maximizar el dinero total para la inversión

Análisis del Problema

Este problema lo podemos modelar como un grafo de n vértices, donde cada vértice es un inversor y es adyacente a todo inversor que lo restringe a participar. Además, va a existir una función $f(x)$ que dado un inversor te devuelva la cantidad de dinero que este prometió aportar. Dado este modelo, el problema se resuelve al maximizar el dinero generado a partir de vértices independientes, es decir, aquellos vértices donde no hay un eje entre ellos.

Un ejemplo es el que se muestra en la derecha, donde la solución es $\{ B, E \}$ con $f(B) + f(E) = 15 + 1 = 16$.

Podemos ver otros conjuntos, pero todos con menor cantidad de dinero, como por ejemplo el conjunto más grande $\{ A, C, D \}$ con $f(A) + f(C) + f(D) = 5 + 6 + 4 = 15$.



i	f(i)
A	5
B	15
C	6
D	4
E	1

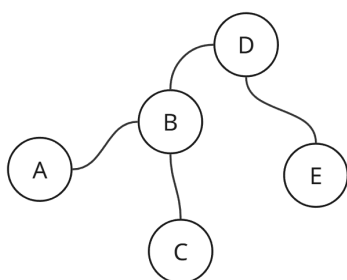
Este problema, así como lo modelamos, se lo conoce popularmente como Maximum Weight Independent Set (MWIS)

Como ya vimos en el curso, Maximum Independent Set (MIS) es NP-Hard. Teniendo esto en mente, podemos reducir polinomialmente Independent Set a este problema al hacer que la función $f(x)$ devuelva el mismo valor para todo vértice. En otras palabras, que cada inversor tenga la misma cantidad de dinero.

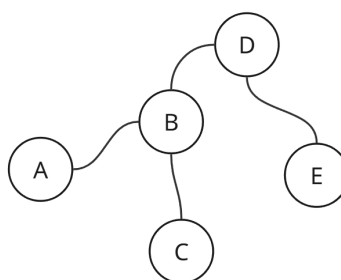
La solución dada, será la solución del Independent Set. Si pusimos los pesos en 1 en cada vertice, entonces el peso máximo será equivalente a la cantidad de vértices elegidos.

A continuación se muestra un ejemplo:

INDEPENDENT SET



WEIGHT INDEPENDENT SET



i	f(i)
A	1
B	1
C	1
D	1
E	1

Cómo podemos reducir polinomialmente un problema NP-Hard a nuestro problema, entonces el mismo también es NP-Hard. Esto demuestra que, para este problema, no se conoce algoritmo polinomial que dé una solución óptima para cualquier instancia de manera eficiente.

Algoritmos

Algoritmo 1: ratio dinero/conflictos

La idea de este algoritmo es asignarle a cada uno de los n inversores, un “ratio” dependiente del dinero que aportan y los conflictos que se generan al incluirlos en el proyecto. Por lo tanto, a cada inversor i le asignaremos un ratio r_i con la siguiente fórmula:

$$r_i = \text{dinero}_i \div (\text{conflictos}_i + 1)$$

Notar el incremento en una unidad al denominador para evitar una posible división por cero.

Una vez que cada inversor i cuente con su r_i correspondiente, los ordenamos según el ratio de manera descendente, y vamos eligiendo de manera greedy el próximo inversor que no haya sido evaluado, y que no tenga conflictos con los inversores elegidos previamente. De esta manera intentamos elegir los inversores más valiosos que no generen conflictos entre sí.

Pseudocódigo

```
Ratio_Inversores(inversores, conflictos):  
    ratios = []  
    para cada inversor en inversores:  
        cant_conflictos = calcular_conflictos(conflictos, inversor)  
        r = inversor.dinero/(cant_conflictos + 1)  
        ratios.append((inversor, r))  
    ratios = ordenar_descendente(ratios)  
    seleccionados = []  
    ganancia = 0  
    para cada inversor, r en ratios:  
        si no tiene conflictos con nadie de seleccionados:  
            seleccionados.append(inversor)  
            ganancia += inversor.dinero  
    retornar seleccionados, ganancia
```

La complejidad temporal de este algoritmo es de $O(n^2)$, ya que en el peor de los casos debemos comparar a cada inversor con el resto al calcular los ratios, y al verificar que no tenga conflictos con otros inversores ya seleccionados.

La complejidad espacial es de $O(m + n)$, siendo m el número de conflictos, ya que este es el tamaño del grafo de conflictos.

Este algoritmo no es óptimo, y podemos encontrar un contraejemplo para probarlo. Supongamos que tenemos 3 inversores A, B y C. Sus posibles aportes están dadas de la siguiente manera:

A = \$2000, B = \$1100, C = \$1000

Ahora supongamos que A genera conflictos con B y con C. B y C no tienen conflictos entre sí.

Calculando el ratio r para cada inversor tenemos:

$$r_a = 2000/(2 + 1) = 666,66$$

$$r_b = 1100/(1 + 1) = 550$$

$$r_c = 1000/(1 + 1) = 500$$

Por lo tanto, la lista de ratios ordenada de manera descendente queda de la siguiente manera: $[r_a, r_b, r_c]$.

De manera Greedy se van a ir evaluando los ratios de mayor a menor. El primer evaluado es A que es seleccionado como parte del resultado porque no genera conflicto con ningún otro inversor que ya haya sido seleccionado (esto es obvio ya que es el primer evaluado). Por lo tanto, nuestra ganancia hasta el momento es de \$2000. Ni B ni C van a ser considerados parte del resultado final porque ambos generan conflictos con A, que ya fue seleccionado previamente. Por lo tanto, la ganancia final es de \$2000, y el único seleccionado es A.

Evidentemente esta solución no es la óptima. Eligiendo a B y a C, conseguimos una ganancia de \$2100. Sin embargo, podemos calcular una relación de aproximación $\rho(n)$.

La relación de aproximación está definida de la siguiente manera:

Un algoritmo A que resuelve un problema de aproximación P tiene una relación de aproximación $\rho(n)$ si para cualquier instancia I de tamaño n, la solución producida por $C = A(I)$ está dentro de un factor de aproximación $\rho(n)$ de la solución óptima C^* .

Dicho factor se calcula de la siguiente manera:

$$\max(C/C^*, C^*/C) \leq \rho(n)$$

Aunque para el análisis de este algoritmo, podemos tomar simplemente

$$C^*/C \leq \rho(n)$$

Ya que al ser un problema de maximización, C^* siempre va a ser mayor o igual a C.

El problema es que no conocemos la solución óptima C^* para cada instancia del problema... Pero podemos aproximarla.

Para cada inversor X que está en C^* pero no en C, tiene que haber un inversor Y en C que fue seleccionado antes y tiene conflictos con X. Cada inversor en C puede bloquear a lo sumo G inversores de C^* , siendo G el grado máximo del grafo de conflictos. En el peor de los casos, $G = n - 1$, significando que un inversor tiene problemas con todo el resto de inversores.

Si Y bloquea a X, podemos asegurar que

$$Y.dinero / (\text{conflictos}(Y) + 1) \geq X.dinero / (\text{conflictos}(X) + 1)$$

De lo contrario, se hubiera evaluado a X antes. Por lo tanto

$$Y.dinero \geq X.dinero * (\text{conflictos}(Y) + 1) / (\text{conflictos}(X) + 1)$$

A su vez,

$$X.dinero * (\text{conflictos}(Y) + 1) / (\text{conflictos}(X) + 1) \geq X.dinero / (G + 1)$$

Ya que $(\text{conflictos}(Y) + 1) \geq 1$, y $(\text{conflictos}(X) + 1) \leq G + 1$

Por lo tanto

$$Y.dinero \geq X.dinero/(G + 1)$$

Esto es así para todos los Y que bloquean un X óptimo, por lo que podemos generalizar que $C \geq C^*/(G + 1)$, o lo que es equivalente, $C(G + 1) \geq C^*$, y nuestro algoritmo es un $(G + 1)$ - algoritmo de aproximación.

Evidentemente esta puede llegar a ser una muy mala aproximación para instancias cuyos grafos de incompatibilidad resultan en grados máximos muy altos (aproximando a n). Pero para grafos con grados máximos lejanos a n, este resulta un buen límite superior. Adicionalmente, un mayor grado en cierto vértice lo penaliza a la hora de calcular su ratio, por lo que es probable que, aunque a priori tengamos un G elevado, el inversor con dicho grado no sea parte de nuestra solución C, y por lo tanto no bloquee inversores de C^* .

Algoritmo 2: randomizado Montecarlo

En este algoritmo se elige iterativamente un inversor disponible, es decir, un inversor que no genere conflictos con uno previamente seleccionado. Se lo suma a la lista de inversores elegidos, y se suma su ganancia. Una vez seleccionado, se eliminan a los inversores conflictivos con el elegido de los inversores disponibles, para asegurarnos que no estemos eligiendo a un inversor inválido en una próxima iteración.

Al obtener una solución parcial a la instancia, se evalúa si es la mejor encontrada hasta el momento, y en caso de ser así se la elige. Este proceso se repite un número preestablecido de iteraciones, intentando minimizar el error causado por elegir aleatoriamente soluciones no muy buenas al problema.

Pseudocódigo

```
Montecarlo(inversores, dinero, conflictos, iteraciones):
    mejor_seleccion = []
    mejor_valor = 0
    durante I iteraciones:
        seleccion_actual = []
        valor_actual = 0
        inversores_disponibles = todos los inversores
        mientras haya inversores disponibles:
            inversor_elegido = un inversor disponible al azar
            seleccion_actual.add(inversor_elegido)
            valor_actual += dinero[inversor_elegido]
            inversores_a_eliminar = todos los conflictos de inversor_elegido
            por cada inv en inversores_a_eliminar:
                sacar a inv de inversores_disponibles
        si valor_actual > mejor_valor:
            mejor_valor = valor_actual
            mejor_seleccion = seleccion_actual
    return mejor_seleccion, mejor_valor
```

La complejidad temporal de este algoritmo es de $O(J \cdot n^2)$, siendo J el número de iteraciones elegidas, ya que para cada iteración debe recorrer todos los inversores, y eliminar a aquellos inversores que crean conflictos con el inversor seleccionado.

La complejidad espacial es de $O(n^2)$, ya que en el peor de los casos, cada inversor tiene conflicto con el resto, así que guardaríamos n inversores en el diccionario de conflictos, cada uno con una lista de tamaño n conflictos.

Este algoritmo claramente no es óptimo. Para el caso más sencillo, vamos a imaginar dos inversores A y B, que tienen un conflicto entre sí. A aportaría 10.000\$, mientras que B solo puede poner 5\$. Como la elección es aleatoria, hay un 50% de probabilidades de elegir al inversor B. Este resultado adverso es menos probable cuanto mayor sea el número de ejecuciones, aunque nunca podremos asegurar al 100% un resultado óptimo.

Sin embargo, podemos hacer un análisis de la probabilidad de encontrar el óptimo.

Cualquier instancia de este problema tiene un subconjunto S de soluciones válidas distintas (no necesariamente óptimas). Es decir, soluciones donde no haya dos inversores con conflictos entre sí, y no queden inversores elegibles sin evaluar.

Entonces en cada iteración, la probabilidad de que la solución aleatoria que elegimos sea la óptima es de $1/|S|$. Estamos asumiendo el peor caso posible donde, de todas las soluciones disponibles, una sola compone la ganancia óptima.

Para J iteraciones > 0 , la probabilidad de obtener el óptimo es de $1 - (1 - 1/|S|)^J$.

Lógicamente cuanto mayor sea el subconjunto de soluciones válidas, menos probable va a ser encontrar el óptimo, y cuanto mayor sea el número de iteraciones, más probable será.

Ejemplo:

Inversores = [A, B, C]

conflictos = A-B

dinero = A: 1000, B: 800, C: 900

Óptimo = [A, C], \$1900

Las soluciones posibles son $S = \{[A, C], [B, C]\}$. Notar que [A], [B] y [C] no son soluciones posibles porque el algoritmo va a seguir buscando inversores disponibles, entonces $|S| = 2$.

La probabilidad de encontrar el óptimo en una iteración es $1/|S| = 1/2$

La probabilidad de encontrar el óptimo en I iteraciones es $1 - (1 - 1/|S|)^J = 1 - (1/2)^J$

Con dos iteraciones ya tenemos un 75% de chances de encontrar el óptimo, y con 4 iteraciones la probabilidad se eleva a un 93,75%.

Adicionalmente, podemos establecer un número de iteraciones dependiente del tamaño de la entrada para garantizar una probabilidad mínima de alcanzar el óptimo, no importa la instancia.

En general, si fijamos el número de iteraciones para que sea igual al tamaño de las soluciones posibles $|S|$, tendremos una probabilidad de aproximadamente 63,21% de encontrar el óptimo para cualquier instancia. Esto es porque la función $1 - (1 - 1/x)^x$ converge a $1 - 1/e$ cuando x tiende a infinito. Por otro lado, si fijamos el número de iteraciones en $2|S|$ la probabilidad aumenta al 86,47%, ya que $1 - (1 - 1/x)^{2x}$ converge a $1 - 1/e^2$.

Por supuesto, no podemos saber de antemano la cantidad de soluciones válidas que hay para cada instancia del problema, sin antes buscar la respuesta con fuerza bruta. Evidentemente esto hace redundante utilizar un algoritmo randomizado para aproximar el óptimo, cuando podemos aplicar el enfoque de fuerza bruta directamente.

Sin embargo, está demostrado² que, para cualquier grafo de tamaño n , pueden haber como máximo $3^{n/3}$ sets independientes máximos. Por lo tanto, podemos decir que en el peor caso las soluciones posibles totales son $|S| = 3^{n/3}$. Entonces, si fijamos el número de iteraciones J en $3^{n/3}$, podemos garantizar que cualquier instancia tiene, como mínimo, un 63,21% de chances de alcanzar el óptimo, y con $2 \cdot 3^{n/3}$ se eleva a un 86,47%. Por supuesto, esto eleva la complejidad temporal $O(J \cdot N^2)$ a $O(3^{n/3} \cdot N^2)$, y hace que nuestro algoritmo deje de ser polinomial. De todas formas, nos pareció interesante el análisis para intentar garantizar una probabilidad mínima de encontrar el óptimo para cualquier instancia del problema.

Análisis empírico de las soluciones

Se adjunta el programa utilizado para comparar los dos algoritmos de manera empírica, escrito en python. En todos los casos, las instancias creadas son aleatorias, y se utiliza un valor aleatorio entre 100\$ y 1.000.000\$ para cada inversor. Se crean instancias con densidades de conflictos **0.1, 0.3, 0.5, 0.7 y 0.9**. Esta es la probabilidad de que dos inversores tengan conflictos entre sí, para intentar representar instancias con menores y mayores grados de conflictos.

La comparación se hace 50 veces por densidad por cantidad de inversores, es decir, se comparan los dos algoritmos 250 veces por cantidad de inversores. Se eligieron instancias de 5, 10, 50, 200 y 700 inversores, por lo que en total se hicieron 1250 comparaciones para intentar establecer para qué casos conviene elegir cierto algoritmo por sobre el rival.

También es importante aclarar que se mantuvo fijo el número de iteraciones del algoritmo randomizado en 30 iteraciones, y todos los valores pueden variar notablemente si se elige una cantidad distinta de iteraciones. En específico, se eligieron 30 porque, empíricamente, daba una buena variedad de resultados al ser comparado con el algoritmo Greedy para distintas circunstancias.

5 INVERSORES					
DENSIDAD	RATIO	MONTECARLO	EMPATES	TIEMPO PROMEDIO (RATIO)	TIEMPO PROMEDIO (MONTECARLO)
0,1	0/50	3/50	47/50	0.00000950s	0.00008735s
0,3	0/50	4/50	46/50	0.00000900s	0.00006714s
0,5	0/50	2/50	48/50	0.00001068s	0.00005591s
0,7	0/50	3/50	47/50	0.00001217s	0.00004728s
0,9	0/50	1/50	49/50	0.00001351s	0.00004249s

² En su paper de 1965 "On cliques in graphs", Moon & Moser demuestran que cualquier grafo de tamaño n tiene, como máximo, $3^{n/3}$ cliques máximos. Esto es análogo al problema de Maximal Independent Sets.

TOTAL	0/250	13/250	237/250	0.00001159s	0.00006287s
--------------	-------	--------	---------	-------------	-------------

10 INVERSORES					
DENSIDAD	RATIO	MONTECARLO	EMPATES	TIEMPO PROMEDIO (RATIO)	TIEMPO PROMEDIO (MONTECARLO)
0,1	1/50	17/50	32/50	0.00001895s	0.00016942s
0,3	3/50	19/50	28/50	0.00002587s	0.00012258s
0,5	3/50	16/50	31/50	0.00003374s	0.00010319s
0,7	2/50	13/50	35/50	0.00004283s	0.00008677s
0,9	2/50	6/50	42/50	0.00005042s	0.00006941s
TOTAL	11/250	71/250	168/250	0.00003436s	0.00011027s

50 INVERSORES					
DENSIDAD	RATIO	MONTECARLO	EMPATES	TIEMPO PROMEDIO (RATIO)	TIEMPO PROMEDIO (MONTECARLO)
0,1	33/50	17/50	0/50	0.00029627s	0.00113852s
0,3	22/50	27/50	1/50	0.00079410s	0.00093341s
0,5	20/50	25/50	5/50	0.00120439s	0.00077170s
0,7	8/50	33/50	9/50	0.00161939s	0.00060759s
0,9	8/50	30/50	12/50	0.00193556s	0.00037787s
TOTAL	91/250	132/250	27/250	0.00116994s	0.00076582s

200 INVERSORES					
DENSIDAD	RATIO	MONTECARLO	EMPATES	TIEMPO PROMEDIO (RATIO)	TIEMPO PROMEDIO (MONTECARLO)
0,1	50/50	0/50	0/50	0.00979163s	0.00941410s
0,3	39/50	11/50	0/50	0.02760300s	0.00817126s
0,5	29/50	21/50	0/50	0.04334421s	0.00662739s
0,7	24/50	22/50	4/50	0.06171426s	0.00504368s

0,9	16/50	30/50	4/50	0.07725483s	0.00284654s
TOTAL	158/250	84/250	8/250	0.04394159s	0.00642059s

700 INVERSORES					
DENSIDAD	RATIO	MONTECARLO	EMPATES	TIEMPO PROMEDIO (RATIO)	TIEMPO PROMEDIO (MONTECARLO)
0,1	50/50	0/50	0/50	0.41250890s	0.10556177s
0,3	48/50	2/50	0/50	1.13581090s	0.09422869s
0,5	39/50	11/50	0/50	1.85146978s	0.07920295s
0,7	32/50	18/50	0/50	2.54686002s	0.05721109s
0,9	23/50	26/50	1/50	3.25243478s	0.02622591s
TOTAL	192/250	57/250	1/250	1.83981687s	0.07248608s

Podemos observar que, para instancias con un número pequeño de inversores, el algoritmo randomizado suele superar al greedy, aunque generalmente prevalece el empate. A pesar de que no se utilizó un algoritmo de fuerza bruta para comparar los resultados frente al óptimo, podemos asumir que en la amplia mayoría de los casos, un empate es señal de que ambos algoritmos propuestos llegaron al óptimo, por lo que los dos resultan buenos para casos así.

Pasando a un caso con una cantidad media de inversores (10-50), el algoritmo de Montecarlo empieza a sacar ventaja, sobre todo en instancias con densidades mayores a 0.1, aunque se sigue observando un número significativo de empates, y de instancias específicas donde prevalece el algoritmo Greedy.

Para los casos con muchos inversores (200-700) el algoritmo Greedy es ampliamente superior, excepto para densidades altas (0,7-0,9). Aunque la superioridad viene con un costo: éste algoritmo es notablemente más lento que su competidor, llegando a tardar más de 3 segundos por instancia en comparación.

También resulta interesante notar que el algoritmo greedy tiene un tiempo de ejecución promedio mayor en densidades más grandes, mientras que podemos ver el caso contrario en el algoritmo de Montecarlo. Esto tiene sentido si tenemos en cuenta el funcionamiento de ambos algoritmos: el algoritmo greedy siempre calcula los ratios de todos los inversores y luego debe verificar que cada uno sea compatible con los seleccionados anteriormente, por lo que una densidad de conflictos mayor hace esta comparación más laboriosa. Por otro lado, una mayor cantidad de conflictos le permite al algoritmo de Montecarlo descartar un mayor número de inversores, reduciendo el número de inversores a evaluar.

En conclusión, si tenemos instancias con pocos inversores, cualquiera de las dos opciones es válida. Para instancias con una cantidad mediana de inversores, el algoritmo de Montecarlo es el preferido, y para instancias con muchos inversores, conviene utilizar el Greedy.

Por otro lado, si desconocemos el número de inversores pero sabemos qué tan conflictivo puede ser nuestro grupo, podemos decir que el algoritmo Greedy es favorecido para pocos conflictos, mientras que el de Montecarlo suele ser el mejor para situaciones con muchos conflictos.

Por último, si nos interesa elegir en base al tiempo de ejecución, el algoritmo de Montecarlo suele superar ampliamente en rapidez a su competidor, y esta diferencia solo se acentúa cuanto mayor sea el número de inversores y la cantidad de conflictos entre ellos.

Parte 3: Autómatas finitos

Los siguientes lenguajes corresponden a la intersección (expresado mediante la conjunción “y”) de dos lenguajes más simples (los símbolos de los mismos corresponden a los números 0 y 1) :

lenguaje "a": $\{w \mid w \text{ (tiene dentro suyo la subcadena "1011") y (tiene un número par de "0") } \}$

lenguaje "b": $\{w \mid w \text{ (toda posición impar es un "1") y (termina con 3 "1") } \}$

A continuación se va a presentar cada uno de los cuatro lenguajes simples y posteriormente los dos lenguajes más complejos.

Por cada lenguaje, se va a detallar:

- Una definición por comprensión.
- Un autómata en forma formal³.
- Una representación gráfica de dicho autómata.
- Una expresión regular.
- Un ejemplo paso a paso en el autómata de una cadena que acepta y otra que rechaza.

Lenguajes simples

Contiene subcadena “1011”

Q $\{ A, B, C, D, E \}$

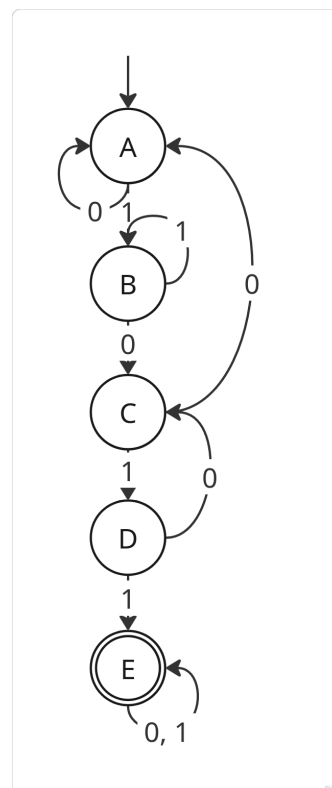
Σ $\{ 0, 1 \}$

δ

	0	1
A	A	B
B	C	B
C	A	D
D	C	E
E	E	E

q_0 A

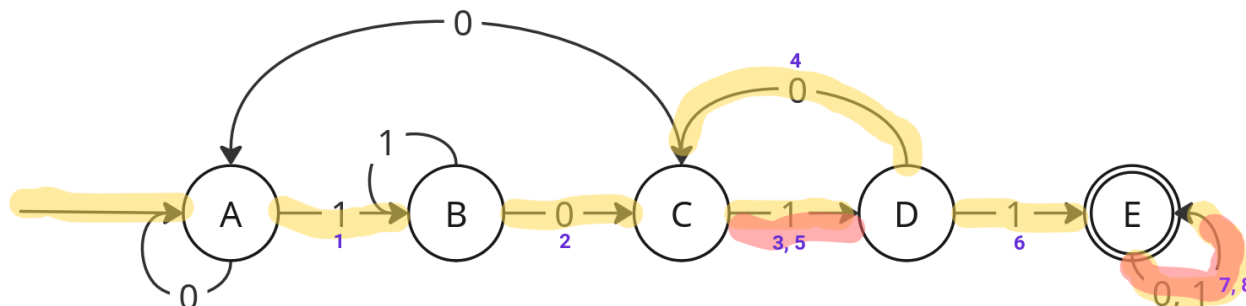
F $\{ E \}$



³ un autómata descrito por su quintupla $(Q, \Sigma, \delta, q_0, F)$, donde estos son 1. el conjunto finito de estados, 2. el alfabeto, 3. una función de transición mostrada como tabla, 4. el estado inicial, 5. un conjunto de estados de aceptación.

Este autómata acepta todas las cadenas que cumplan con esta expresión regular **$.^*1011.^*$** .

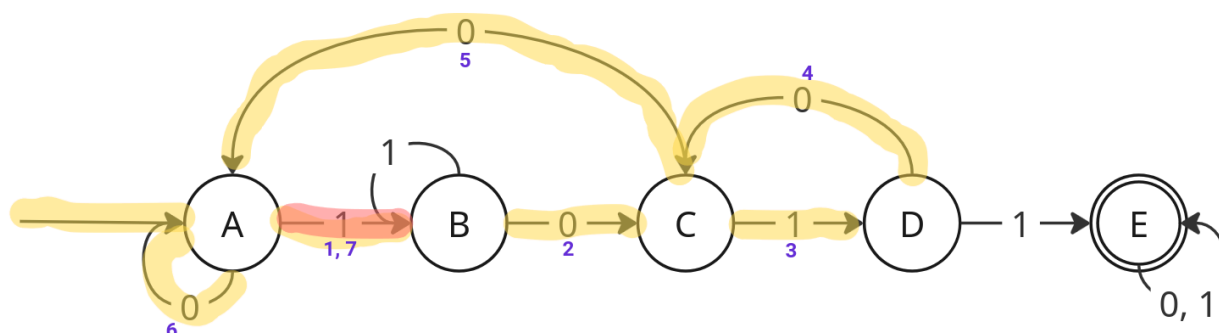
Un ejemplo que cumple es 10101110 el cual pasa por los siguientes estados:



los números en violeta representan los pasos

El autómata determina si acepta o no en el último paso (a menos que se rechace antes), cuyo número coincide con la longitud de la cadena (8). Al terminar en un estado de aceptación (E), la cadena es aceptada.

Una cadena no aceptada es 101010100.



los números en violeta representan los pasos

Se puede apreciar que la cadena finaliza en el estado B, que no es un estado de aceptación. Por lo tanto la cadena es rechazada.

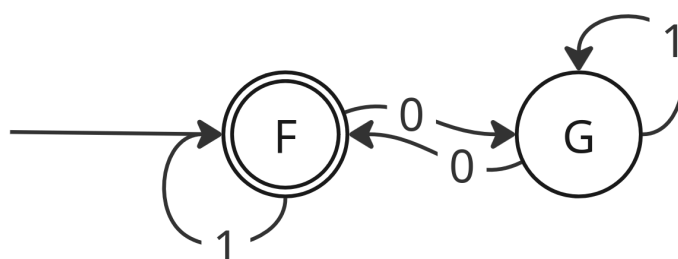
Número par de ceros

Q $\{ F, G \}$

Σ $\{ 0, 1 \}$

δ

	0	1
F	G	F
G	F	G

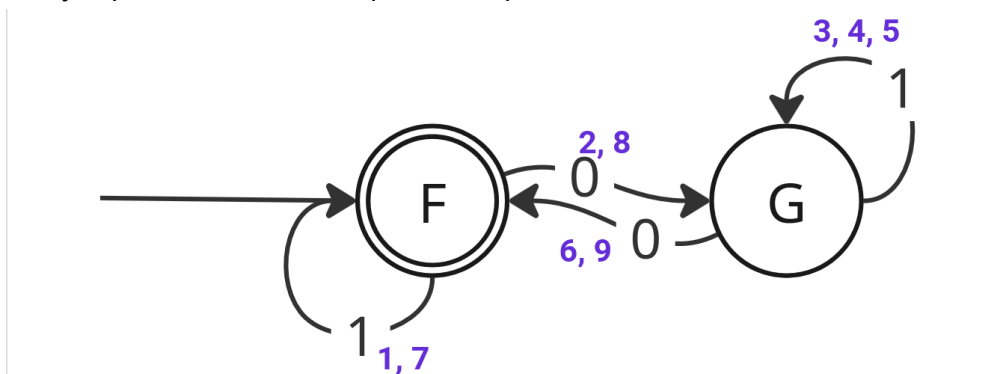


q_0 F

F { F }

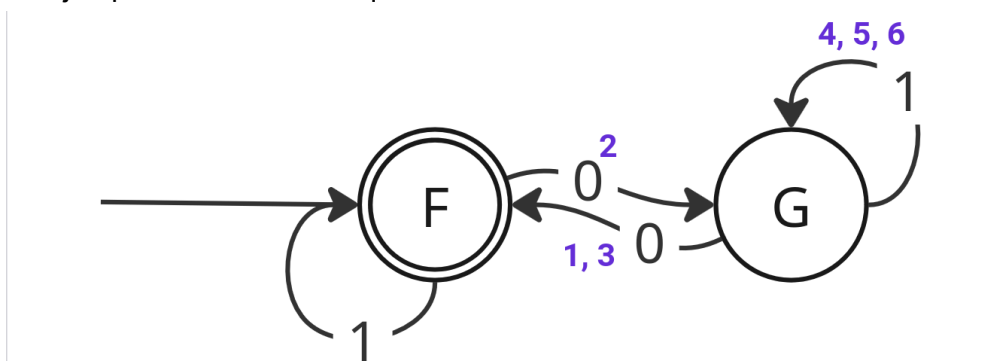
Este autómata acepta todas las cadenas que cumplen la expresión regular **$(1^*01^*01^*)^*$** .

Un ejemplo de una cadena que se acepta es 101110100



Se puede ver que esta cadena finaliza en el paso 9 en el estado F, el cual es un estado de aceptación.

Un ejemplo de una cadena que se rechaza es 000111



El cual termina en el estado G tras el paso 6. Y este estado no es de aceptación.

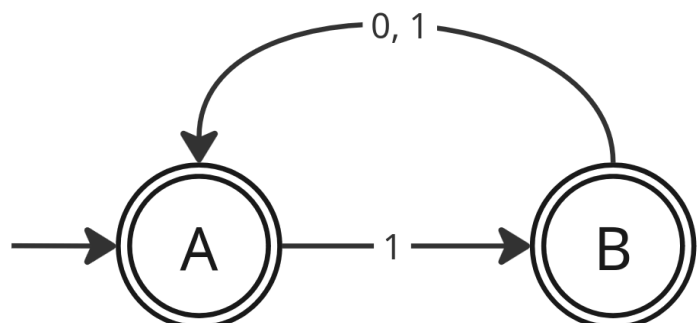
Poseen un **1** en posiciones impares

Q { A, B }

Σ { 0, 1 }

δ

	0	1
A	-	B
B	A	A

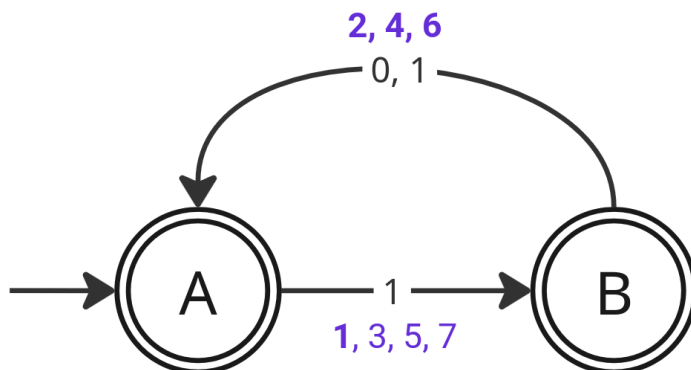


q_0 A

F { A, B }

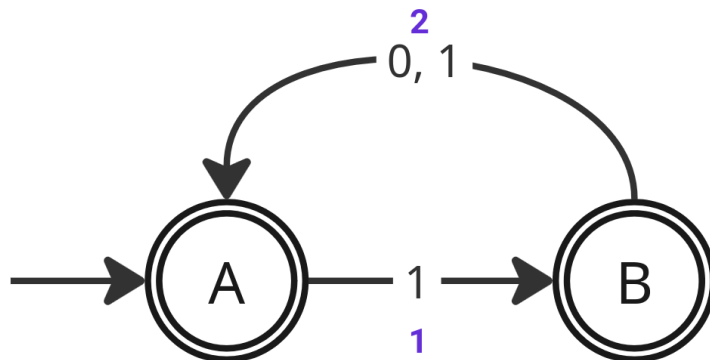
Este autómata acepta todas las cadenas que cumplen la expresión regular $(1(1 \cup 0))^*1^*$.

Un ejemplo de una cadena que acepta es 1011101:



Al finalizar la cadena, se encuentra en un estado de aceptación.

Un ejemplo de una cadena que no es aceptada es 1101111:



Al encontrarse en el paso **3** dentro del estado **A** con un símbolo **0**, el autómata no tiene transición posible y, por lo tanto, se rechaza la cadena.

Termina con tres **1**

Q { E, F, G, H }

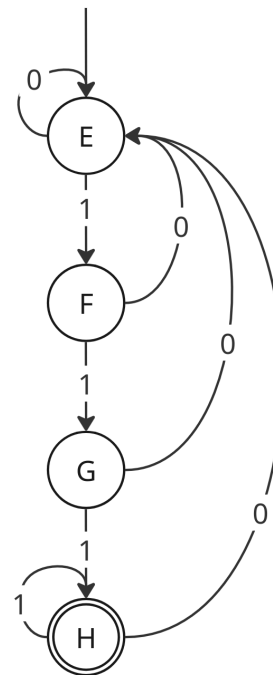
Σ { 0, 1 }

δ

	0	1
E	E	F
F	E	G
G	E	H
H	E	H

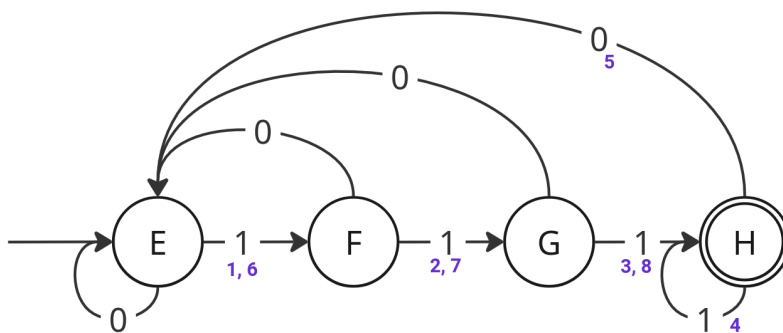
q_0 E

F { H }



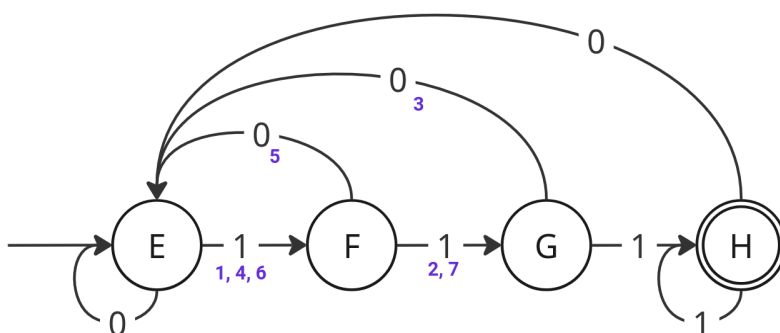
Este autómata acepta todas las cadenas que cumplen la expresión regular **.^{*}111**.

Un ejemplo de una cadena que acepta es 11110111:



Al finalizar la cadena, se encuentra en **H**, un estado de aceptación.

Un ejemplo de una cadena que no es aceptada es 1101011:



Al finalizar la cadena se encuentra en **G**, que no es un estado de aceptación y se rechaza.

Lenguajes Complejos

Para hacer una intersección de dos autómatas finitos deterministas “a” y “b” vamos a utilizar la técnica del producto cartesiano, el cual construye un nuevo AFD a partir de la combinación de todos los pares posibles de estados de dos autómatas, quedando un nuevo autómata de $|Q_a| \cdot |Q_b|$ estados.

Ambos autómatas deberán estar definidos sobre el mismo alfabeto Σ .

Su estado de inicio será aquel estado (q_a, q_b) tal que q_a sea el estado de inicio de “a” y análogo con q_b .

Sus estados de aceptación serán aquellos estados (q_a, q_b) tal que q_a y q_b sean estados de aceptación de “a” y “b” respectivamente.

Su función de transición $\delta((q_a, q_b), x)$ será equivalente a $(\delta_a(q_a, x), \delta_b(q_b, x))$ para todo símbolo x del alfabeto.

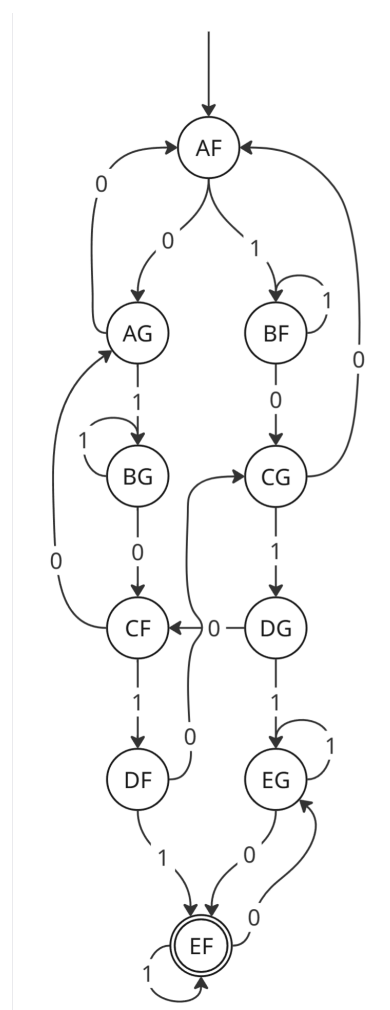
Contiene subcadena “1011” y número par de ceros

Q { AF, AG, BF, BG, CF, CG, DF, DG, EF, EG }

Σ { 0, 1 }

δ

	0	1
AF	AG	BF
AG	BF	BG
BF	CG	BF
BG	CF	BG
CF	AG	DF
CG	AF	DG
DF	CG	EF
DG	CF	EG
EF	EG	EF
EG	EF	EG



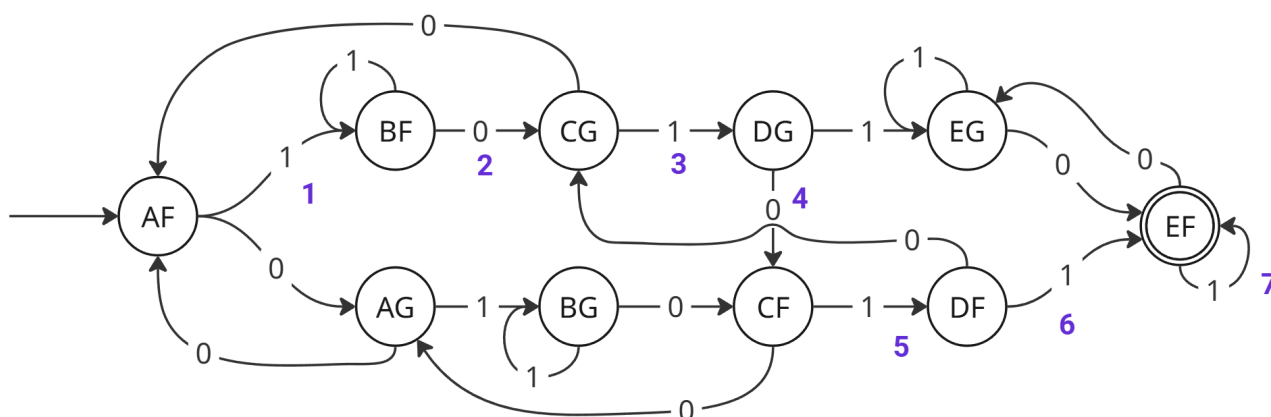
q_0 AF

F { EF }

Este autómata acepta todas las cadenas que cumplan con esta expresión regular

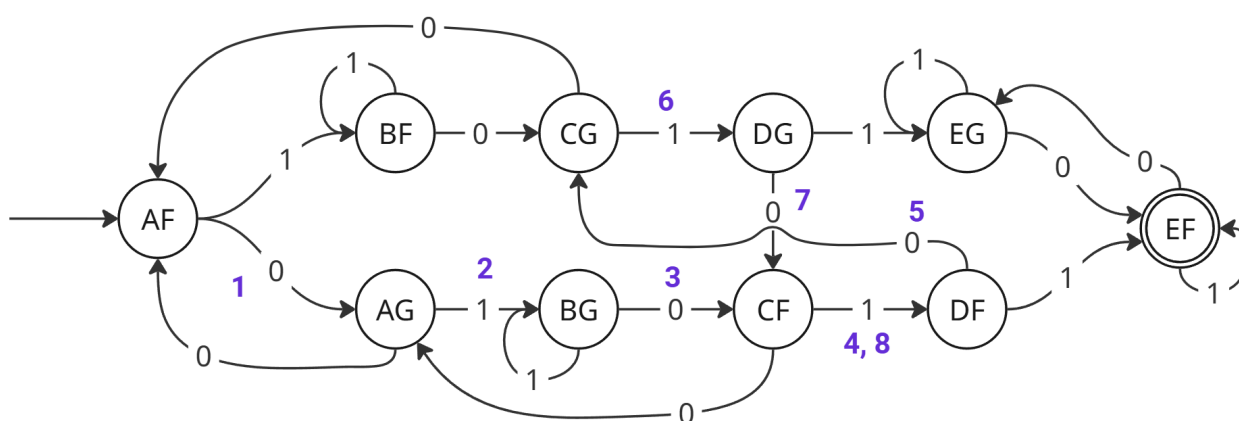
$(. * 1011. *) \cap ((1 * 01 * 01 *) *)$.

Un ejemplo de una cadena aceptada es 1010111



Al finalizar la cadena se encuentra en EF, un estado de aceptación.

Un ejemplo de una cadena que no es aceptada es 01010101 por no contener la cadena 1011



Luego de la iteración, la cadena finaliza en DF, el cual no es un estado de aceptación.

Poseen un 1 en posiciones impares y finalizan con tres 1

Q { AE, AF, AG, AH, BE, BF, BG, BH }

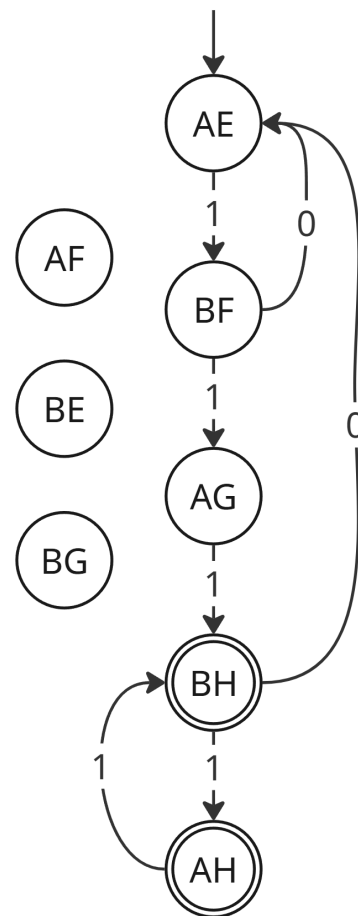
Σ { 0, 1 }

δ

	0	1
AE	-	BF
AF	-	BG
AG	-	BH
AH	-	BH
BE	AE	AF
BF	AE	AG
BG	AE	AH
BH	AE	AH

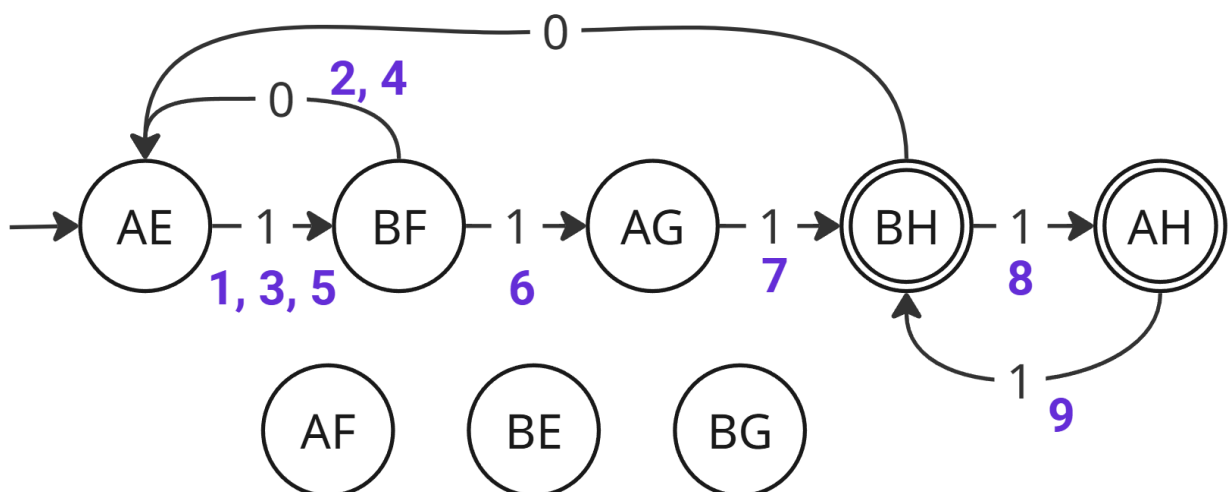
q_0 AE

F { AH, BH }



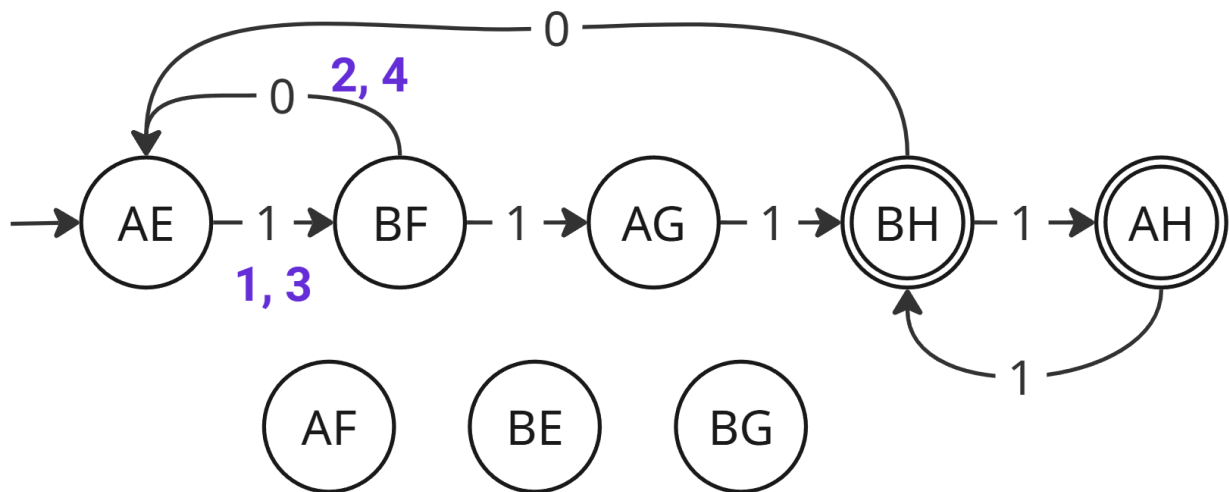
Este autómata acepta todas las cadenas que cumplan con esta expresión regular $((1(1\cup 0))^*1^*)\cap(.^*111)$.

Una cadena que acepta es **101011111**



Esta finaliza en BH, un estado de aceptación.

Un ejemplo de una cadena que no es aceptada es 10100111



Ya que al posicionarse en AE y evaluar un 0, no hay transición posible y, por lo tanto, se rechaza.

Referencias

1. [Resumen Simplex con implementación básica](#)
2. [Código de Simplex en Python](#)
3. [Programación Lineal Entera - Investigación Operativa - FI UNMDP](#)
4. [Teoría de la computación: Intersección de Lenguajes Regulares. curso CC3102. fcfm.](#)
5. [Video corto: Intersección Entre Dos Autómatas.](#)
6. [Maximal independent set](#)
7. [Moon & Moser \(1965\)](#)
8. [Podberezski. V.: Algoritmos de Aproximación - Presentación](#)